Lab 7: Event-Driven Trip Processing Pipeline using AWS Kinesis, Lambda, DynamoDB, and Glue

Lab Summary

This lab demonstrates how to build an **event-driven data processing pipeline** for a ride-hailing company scenario. Trip start and trip end events are streamed through **Amazon Kinesis**, processed using **AWS Lambda**, stored in **Amazon DynamoDB**, and aggregated using **AWS Glue** before being saved to **Amazon S3**.

The pipeline captures trip details when a trip starts and updates them when the trip ends, enabling near real-time insights and analytics.

Prerequisites to Complete Lab

Before starting, ensure you have:

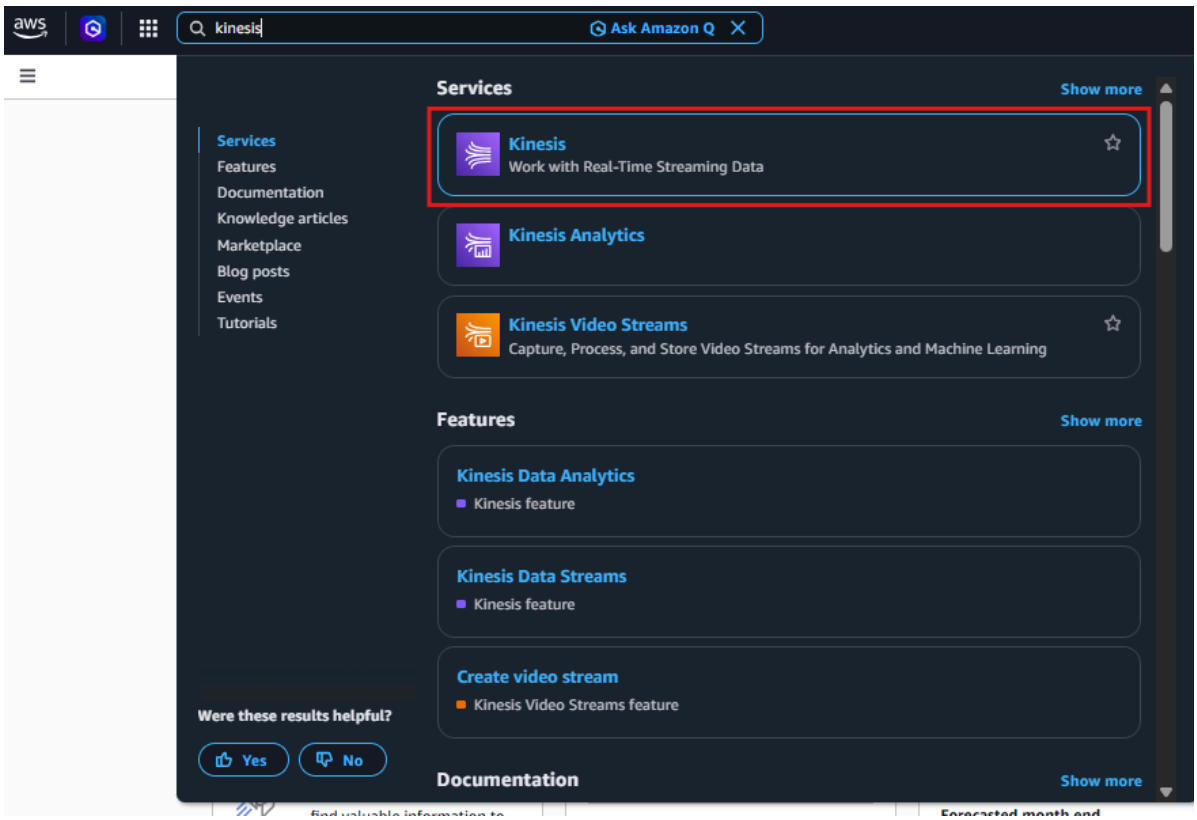
- AWS account with admin or required permissions
- AWS CLI configured (`aws configure`)
- Python 3 installed locally
- Input CSV files: trip start and trip end datasets
- Python producer script to push data to Kinesis
- Basic knowledge of AWS services (Kinesis, Lambda, Glue, DynamoDB)
- IAM roles with permissions:
  - DynamoDB Full Access
  - Kinesis Full Access
  - CloudWatch Logs Full Access

- Glue Console Full Access
- S3 Full Access (for Glue role)

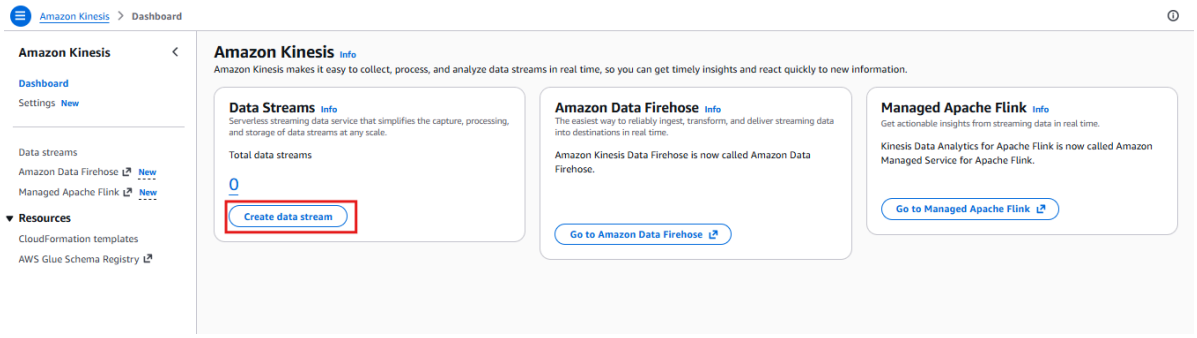
Steps to Complete Lab

Task 1 — Create Kinesis Data Streams

1. Open AWS Console → Search **Kinesis**.

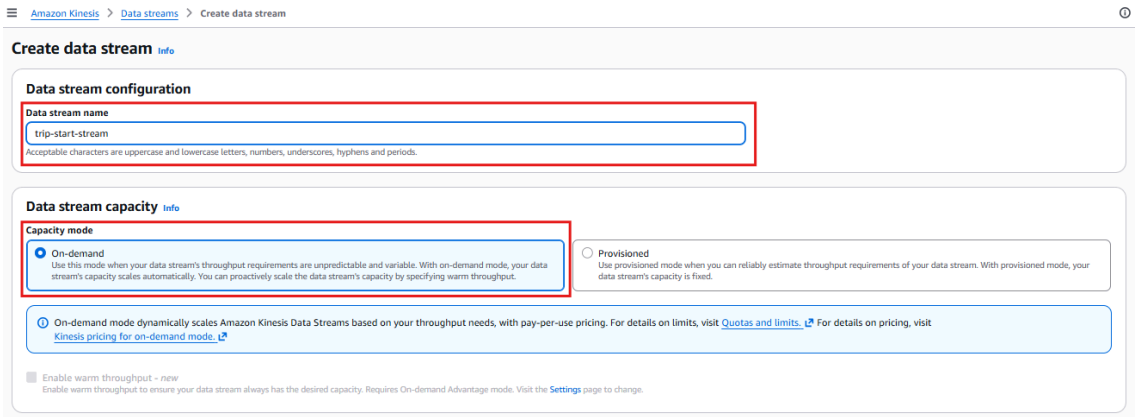


2. Click **Create data stream**.

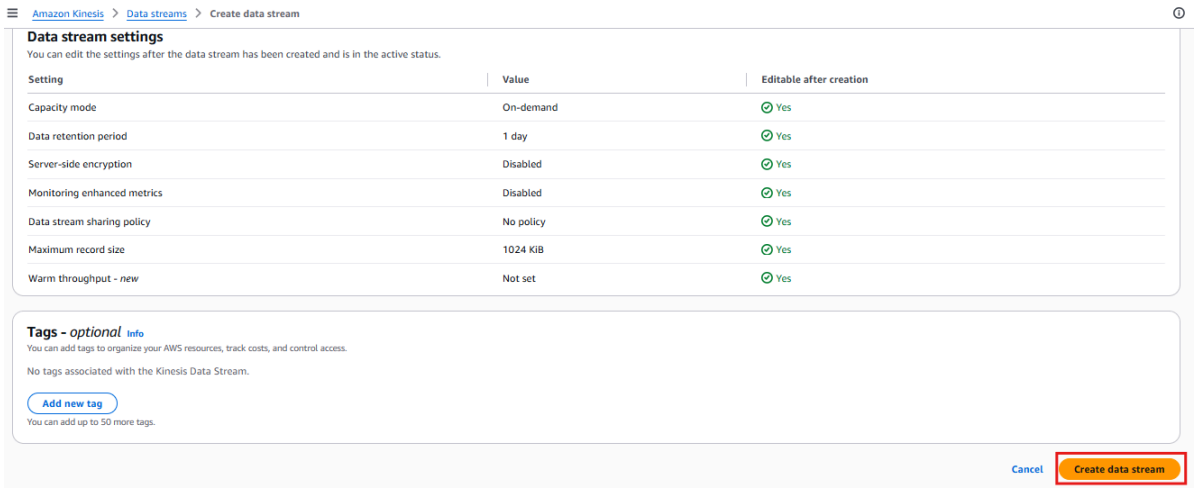


3. Create first stream:

   - Name: `trip-start-stream`
   - Capacity: On-demand

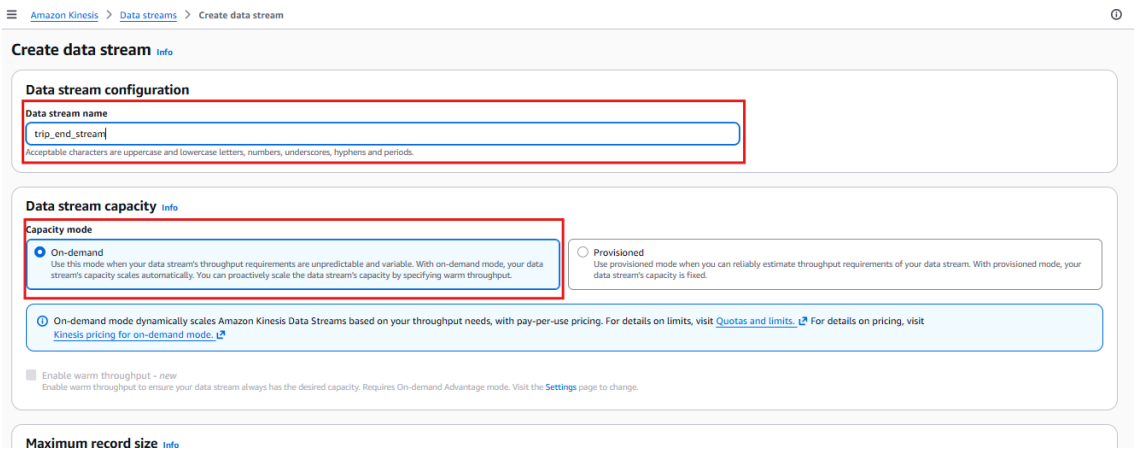


4. Click on create data stream



5. Create second stream:

- Name: `trip-end-stream`
- Capacity: On-demand



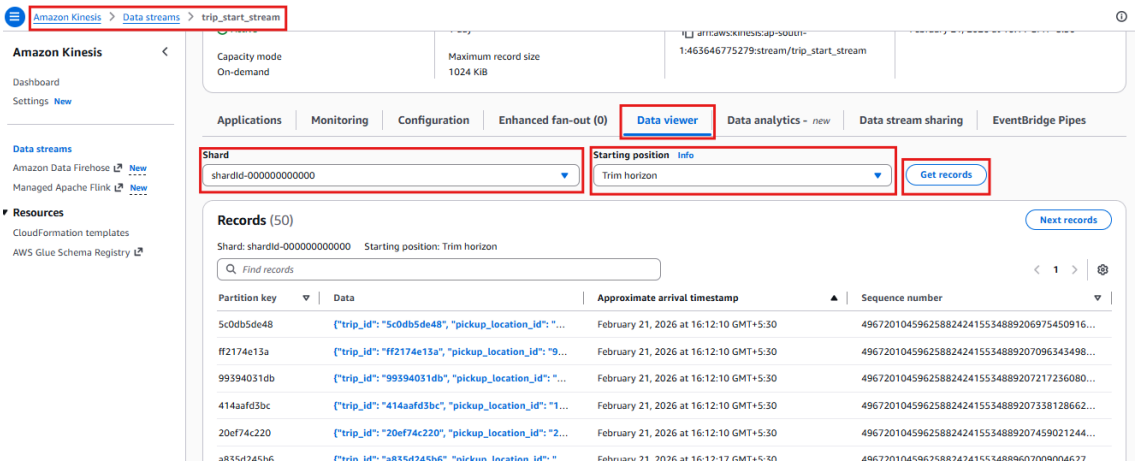
6. Click on create data stream

Task 2 — Run Producer Script Locally

1. Navigate to project directory.
2. Ensure CSV files exist.
3. Run producer script:

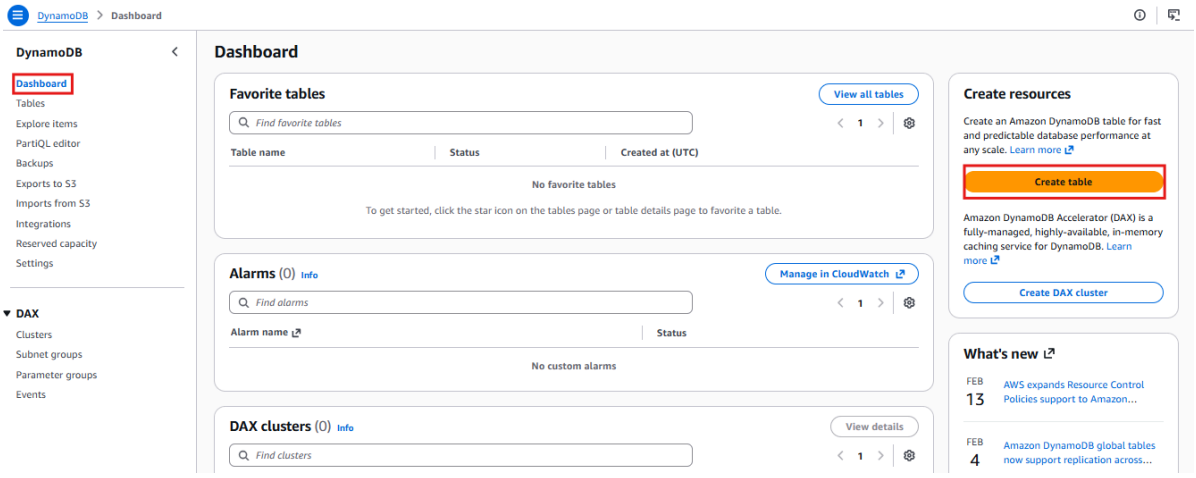
```
py kinesis_trips_producer.py
```

4. Verify data ingestion:
   - Open stream → Data Viewer → Get Records

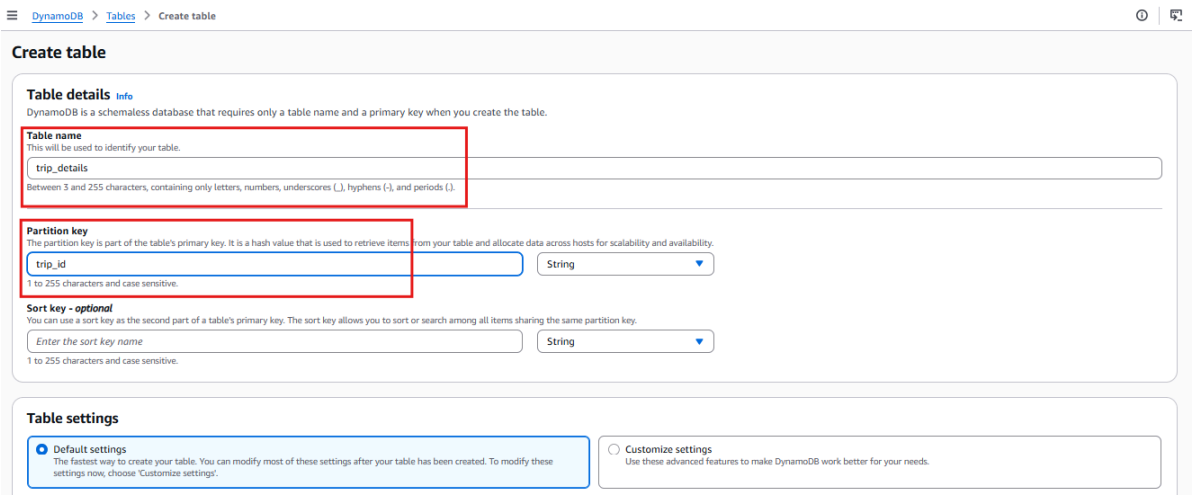


Task 3 — Create DynamoDB Table

1. Open DynamoDB → Create table.



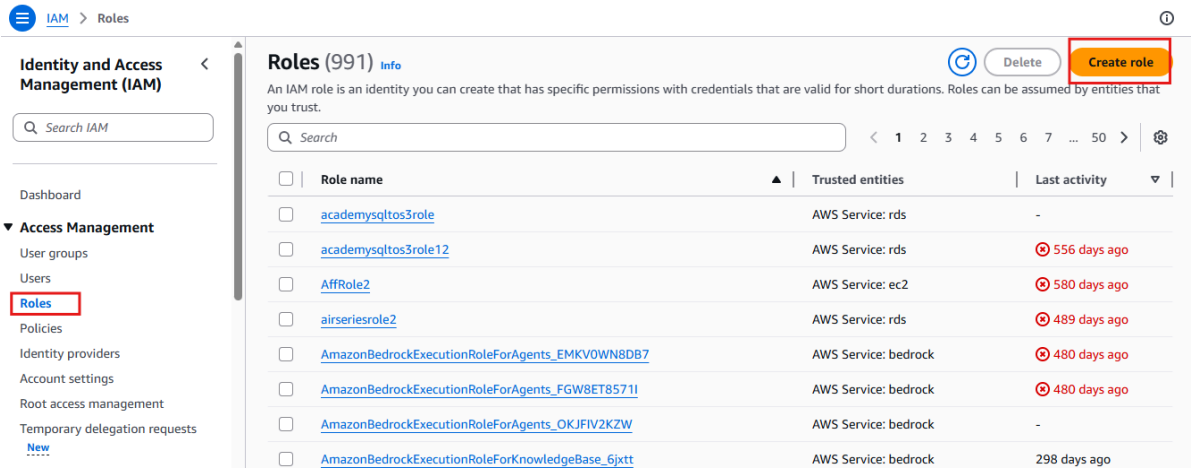
2. Table name: `trip-details`.
3. Partition key: `trip_id`.



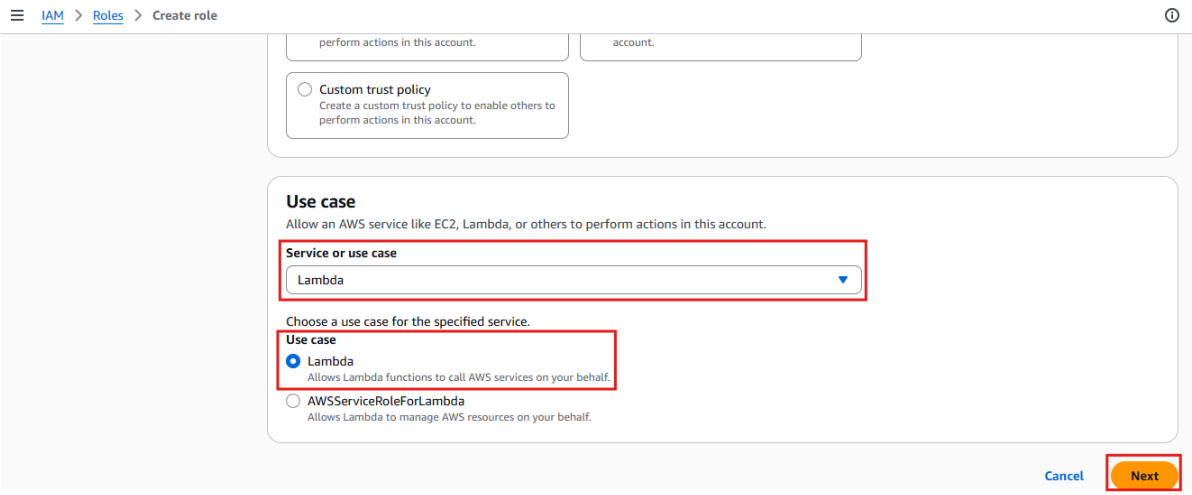
4. Scroll down —→ Click Create table.

Task 4 — Create IAM Role for Lambda

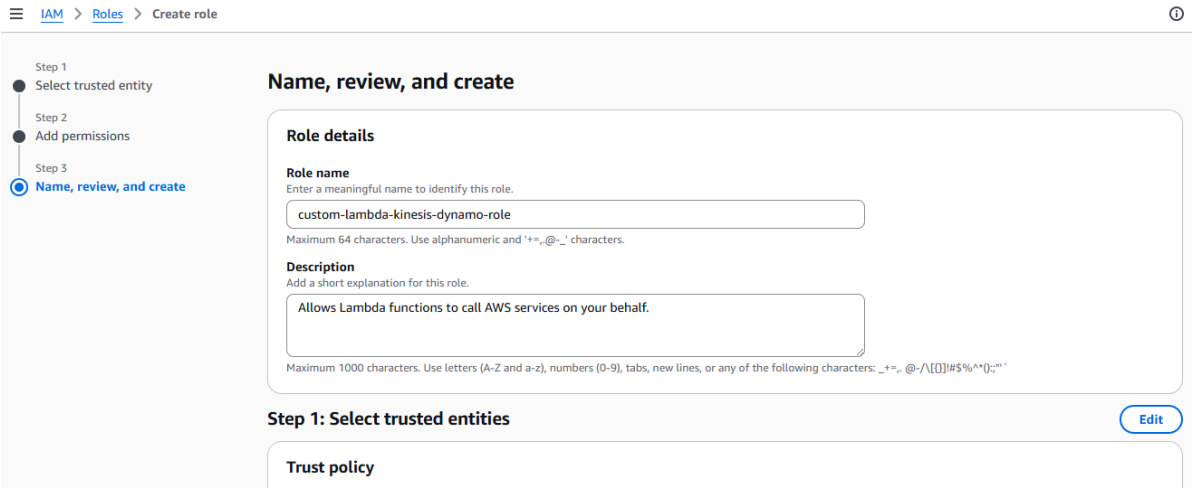
1. Go to IAM → Roles → Create role.



2. Select Use case: Lambda —→ Click Next



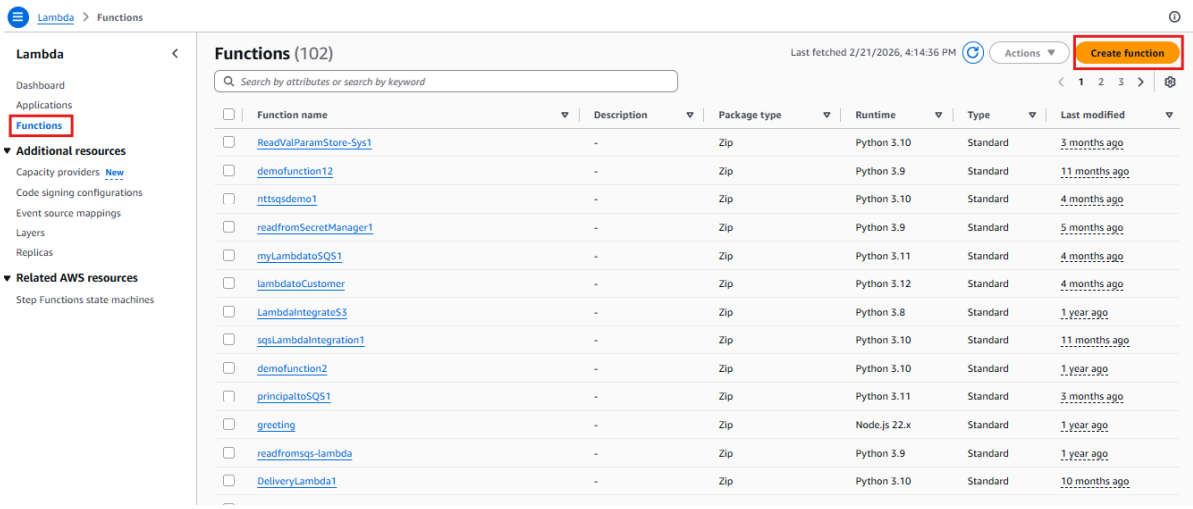
3. Attach policies:
    - DynamoDB Full Access
    - Kinesis Full Access
    - CloudWatch Logs Full Access
    - Glue Console Full Access
4. Give it a name:



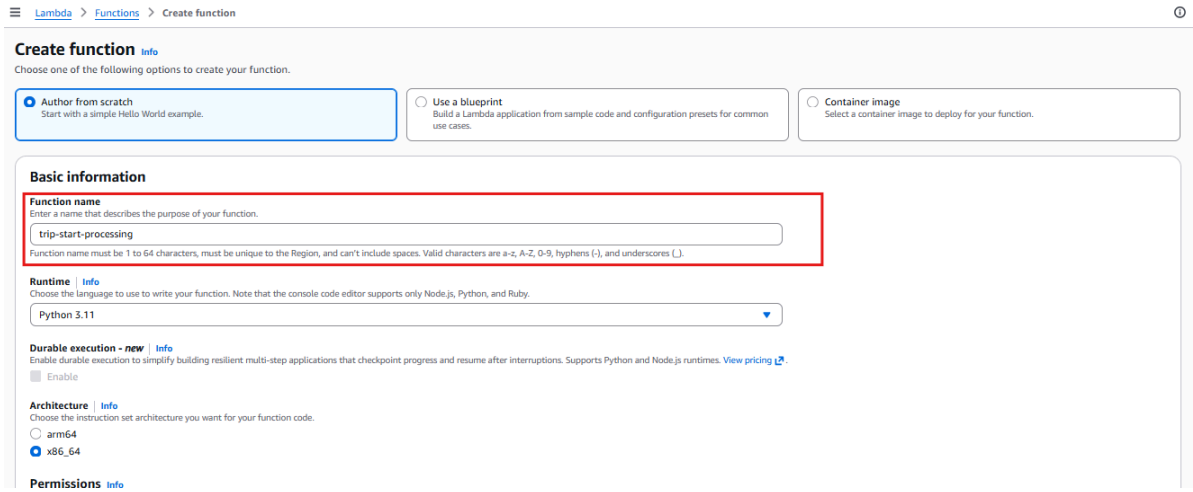
5. Click Create role.

Task 5 — Deploy Lambda Function (Trip Start)

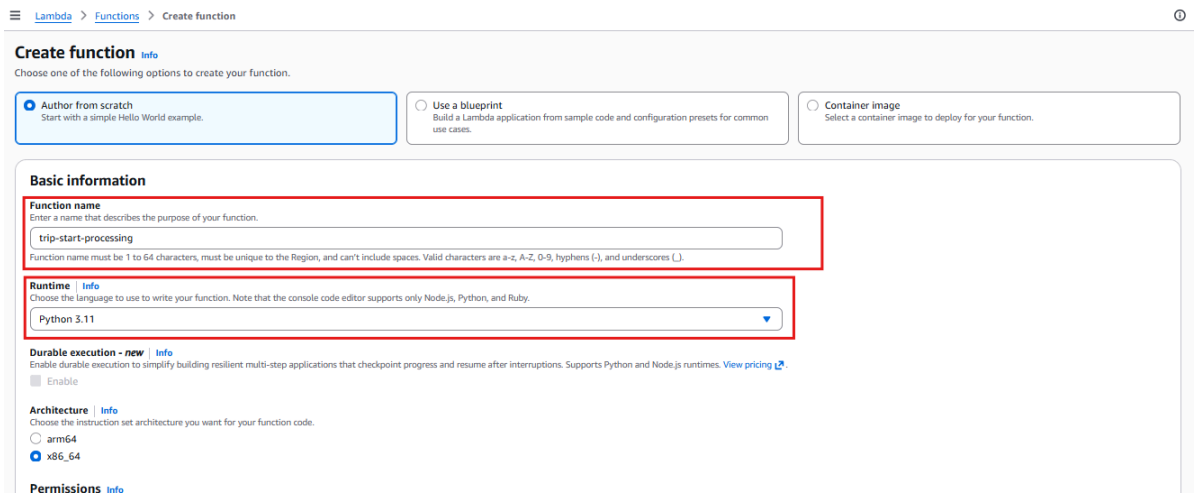
1. Open Lambda → Create function.



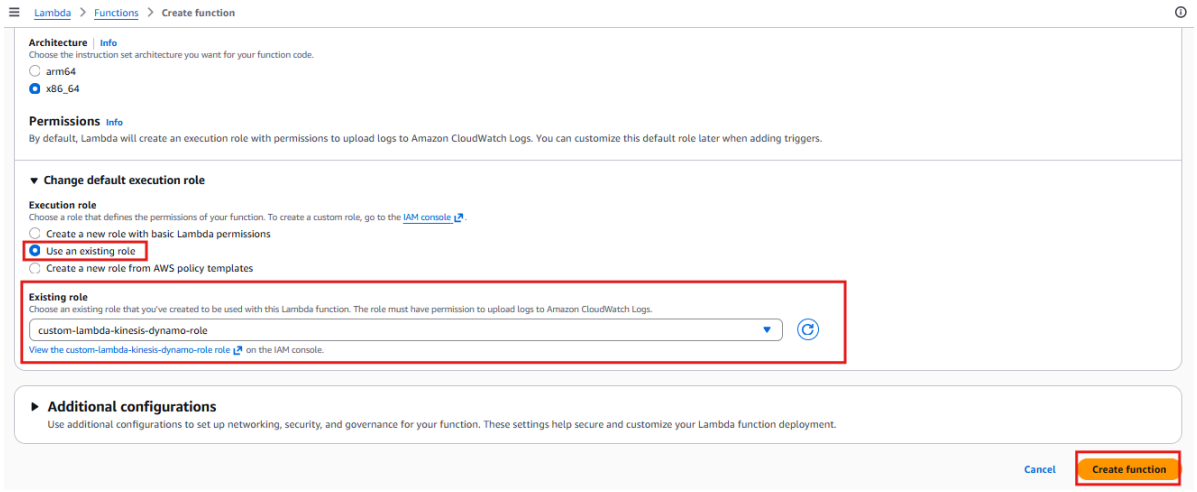
2. Name: `trip-start-processing`.



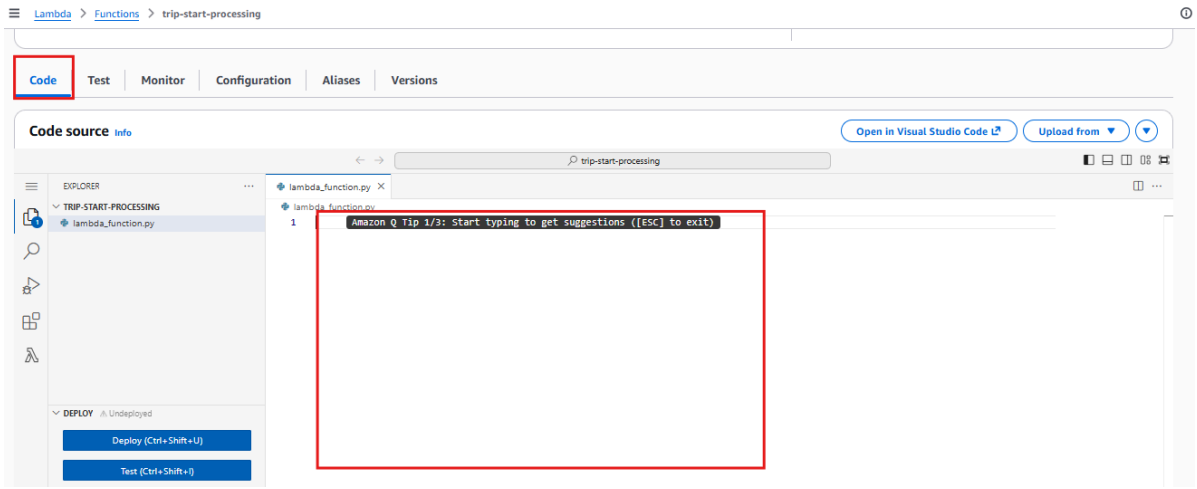
3. Runtime: Python 3.11.



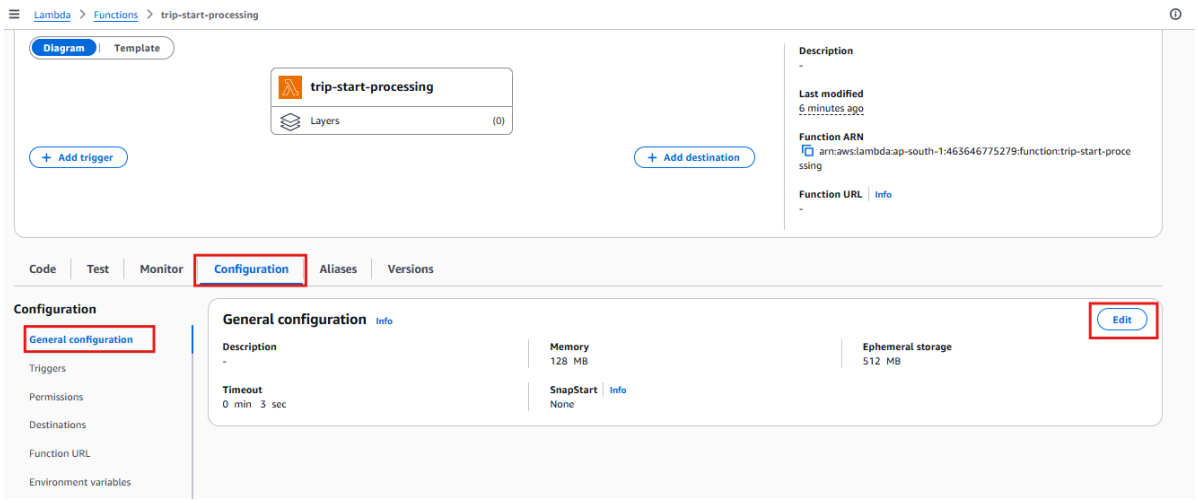
4. Select IAM role created earlier —→ Click on create function



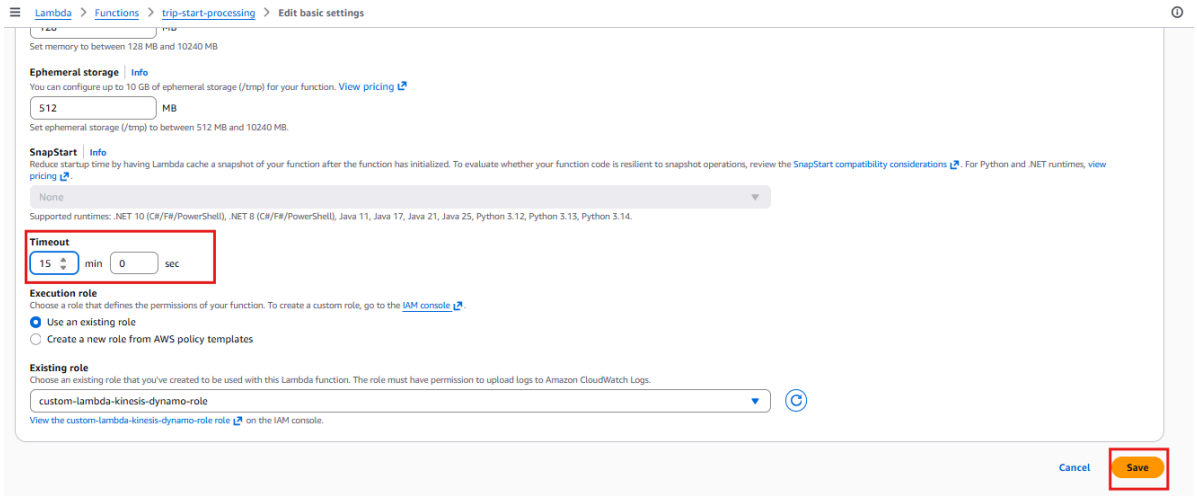
5. Go to code tab —→ Paste lambda_trip_start code.



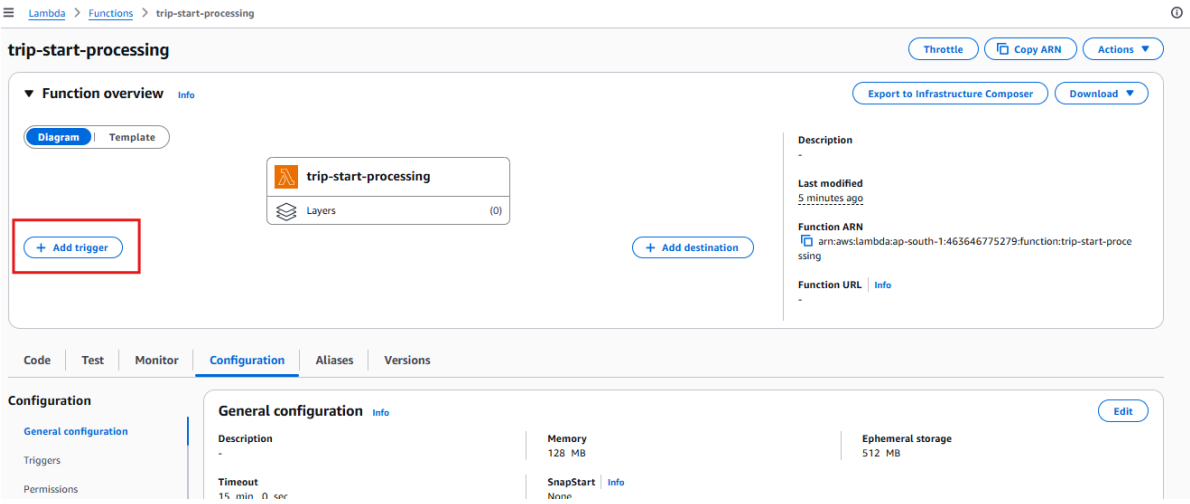
6. Go to configuration tab —→ General configuration —→ Click Edit

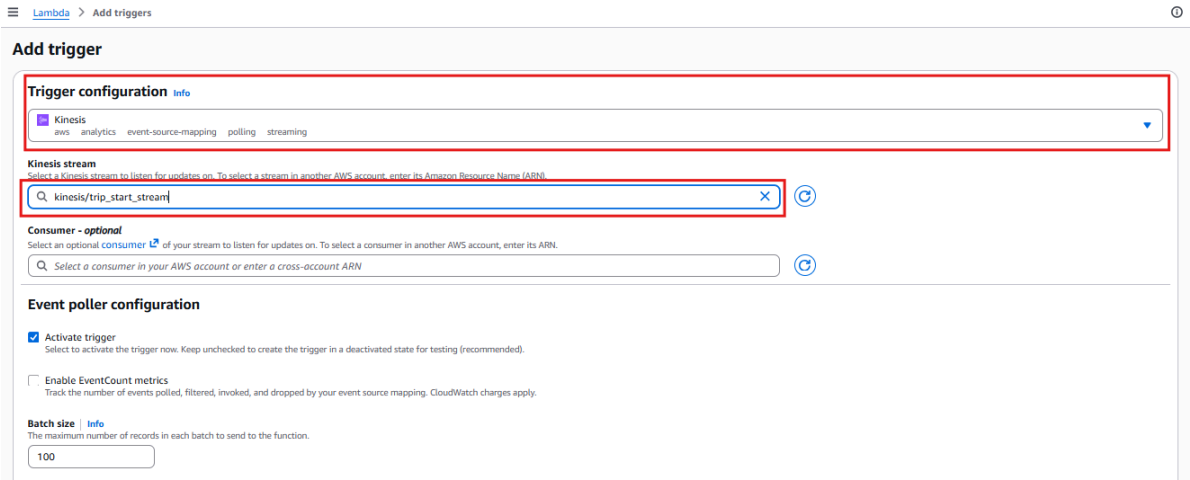


7. Set timeout to **15 minutes**. —→ Click on save



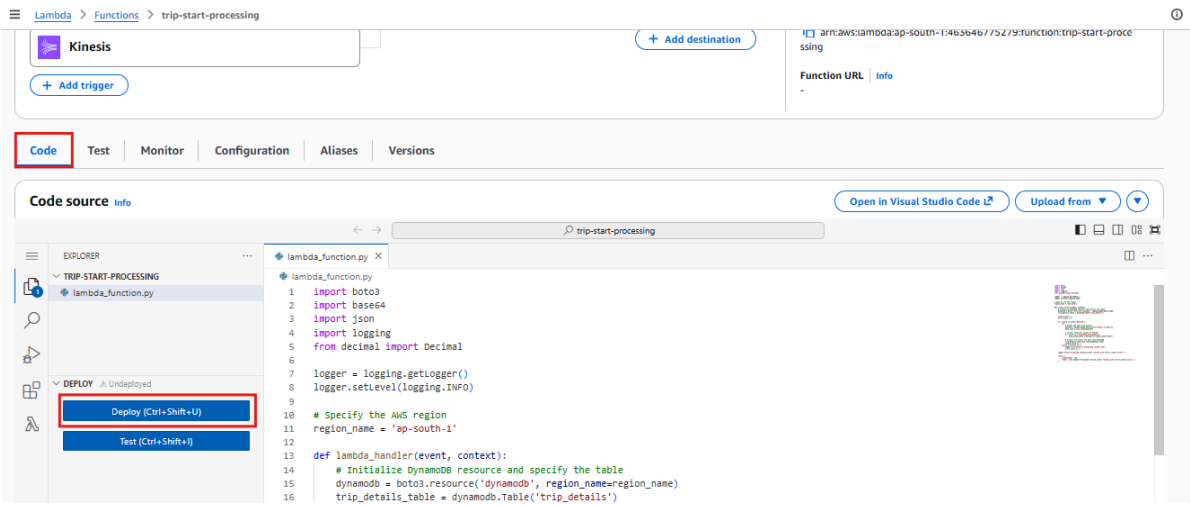
8. Click on Add trigger → Kinesis → select `trip_start_stream`.





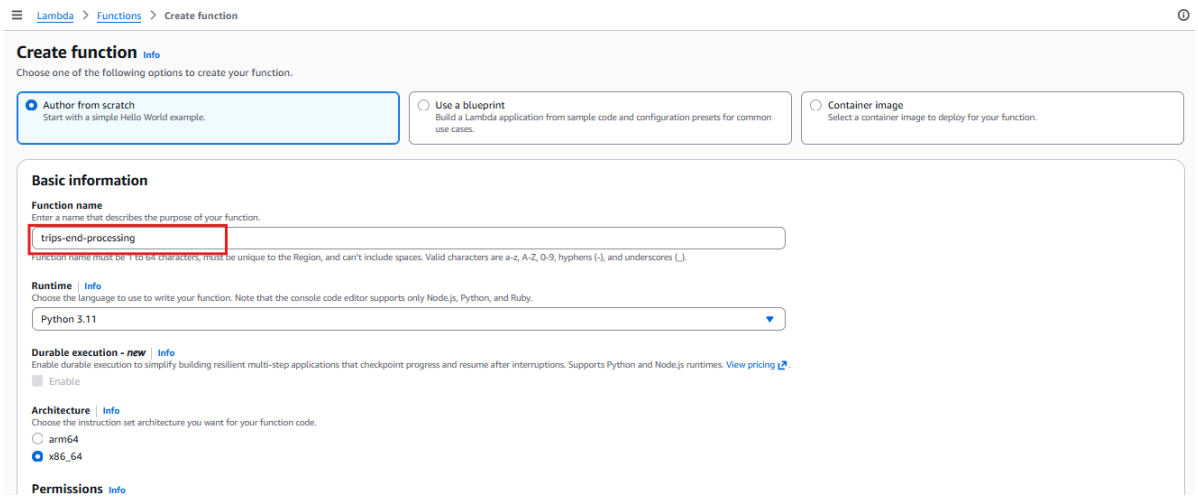
9. Click Add

10. Go back to code tab —→ click on Deploy

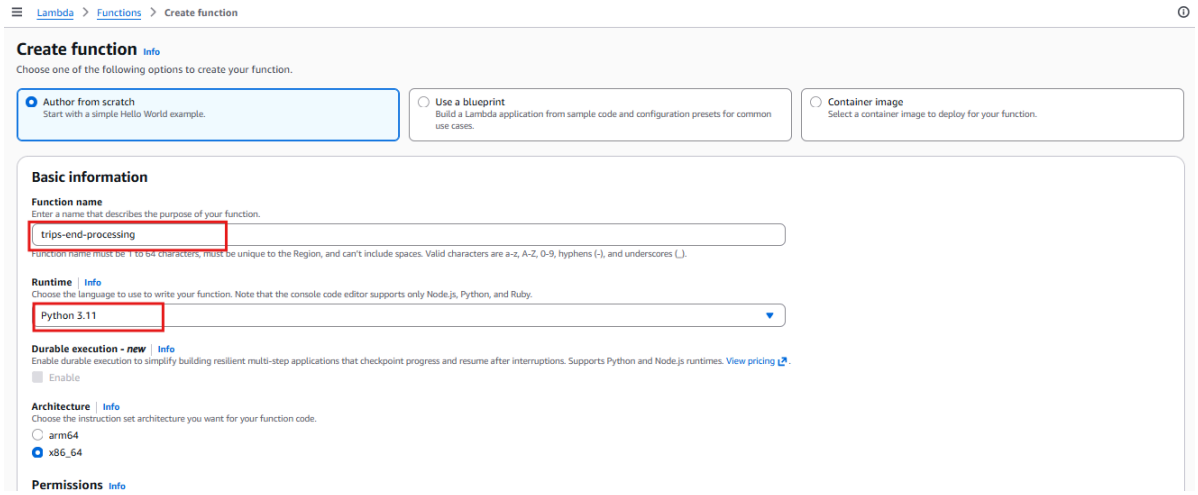


Task 6 - Create & Deploy Lambda Function (Trip End)

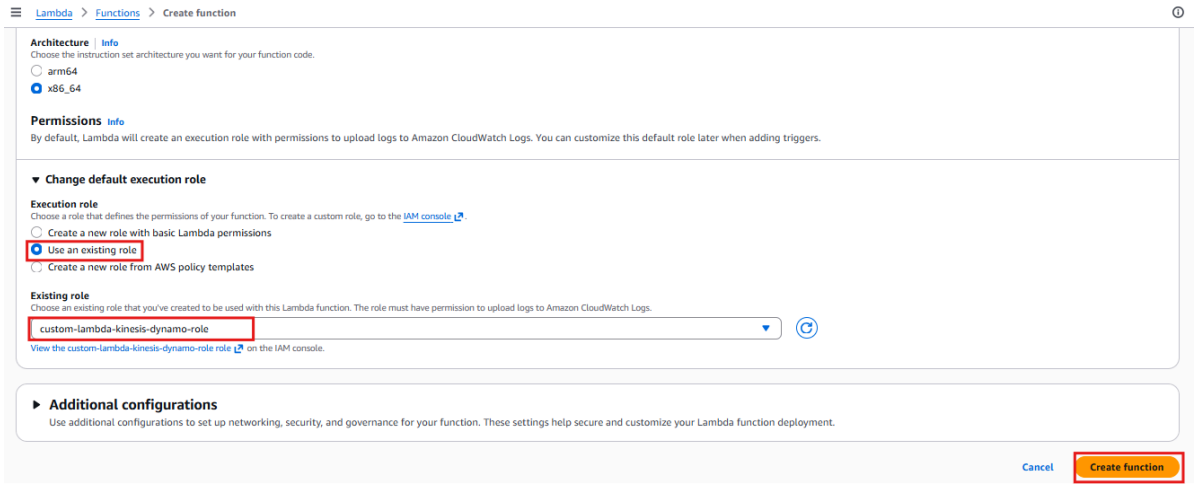
1. Create new function: `trips-end-processing`.



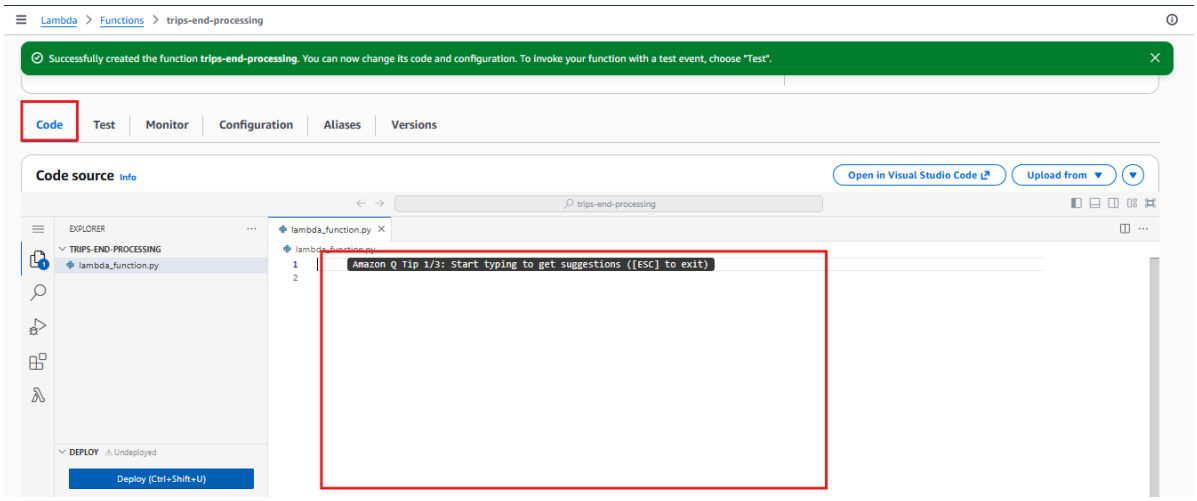
2. Runtime: Python 3.11.



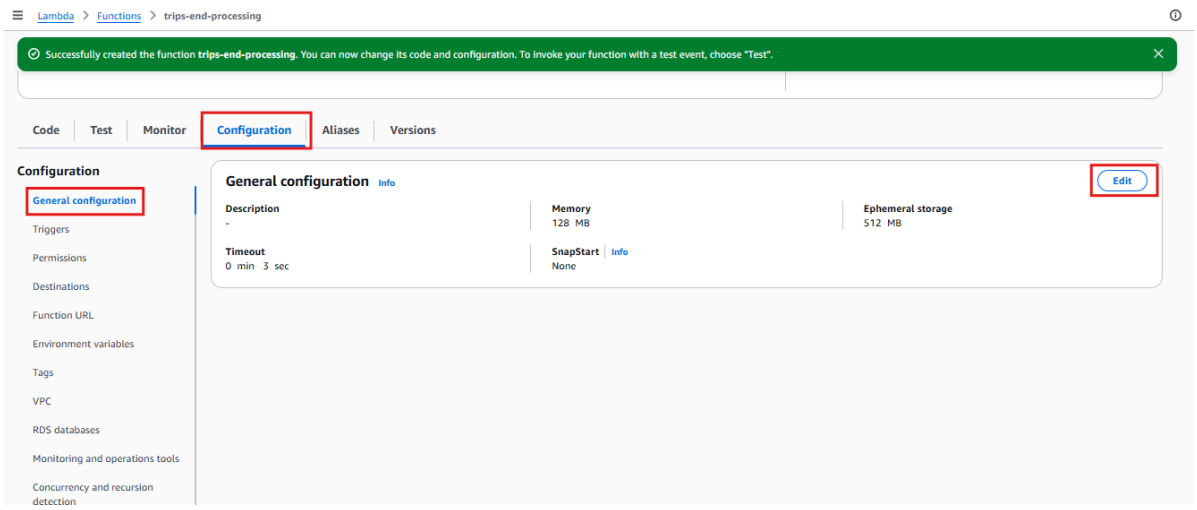
3. Use same IAM role —→ Click create function



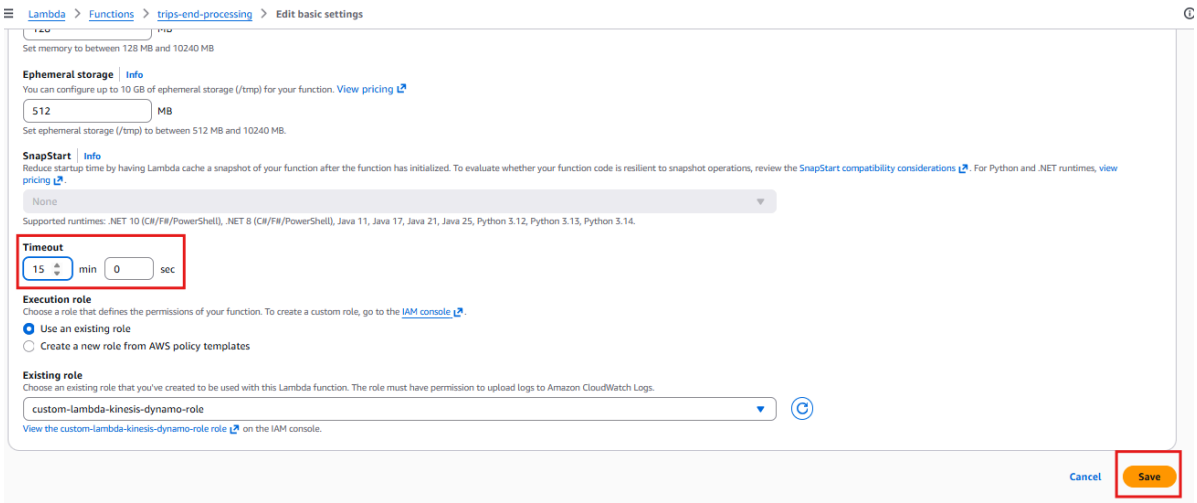
4. Scroll down to code tab —→ Paste lambda_trip_end code.



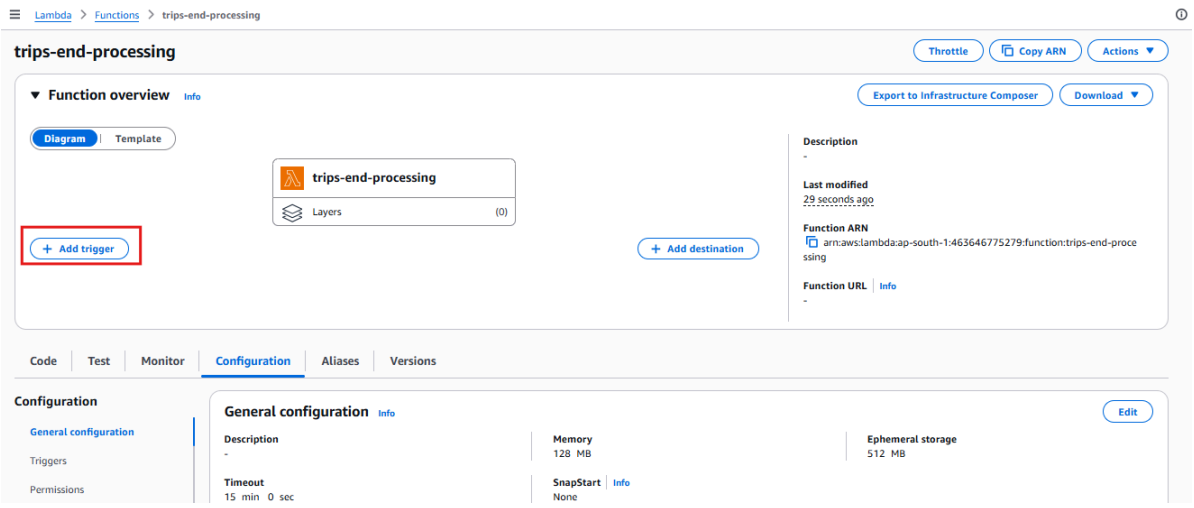
5. Go to configuration —→ General configuration —→ Click Edit



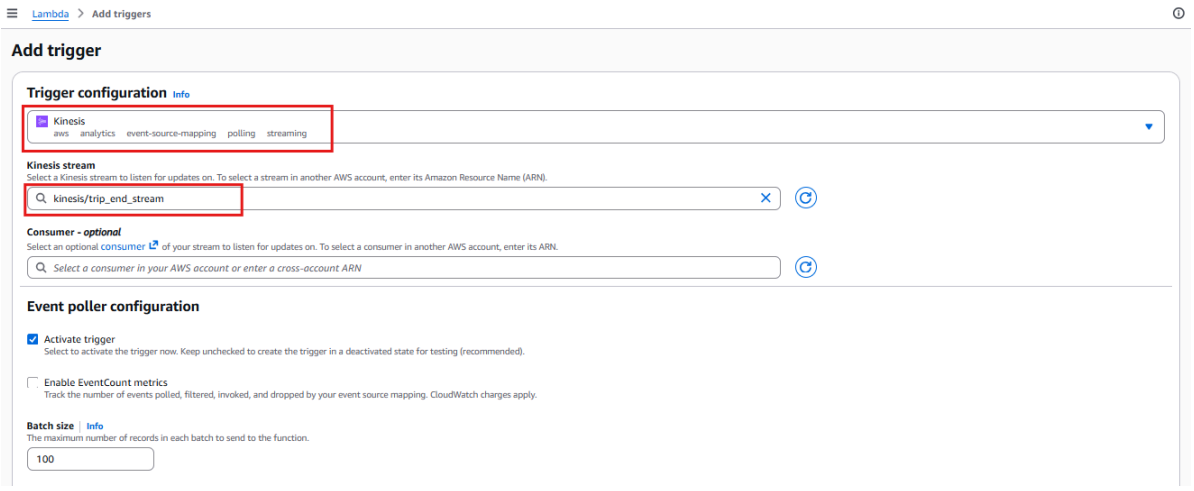
6. Set timeout to 15 minutes. —→ Click save



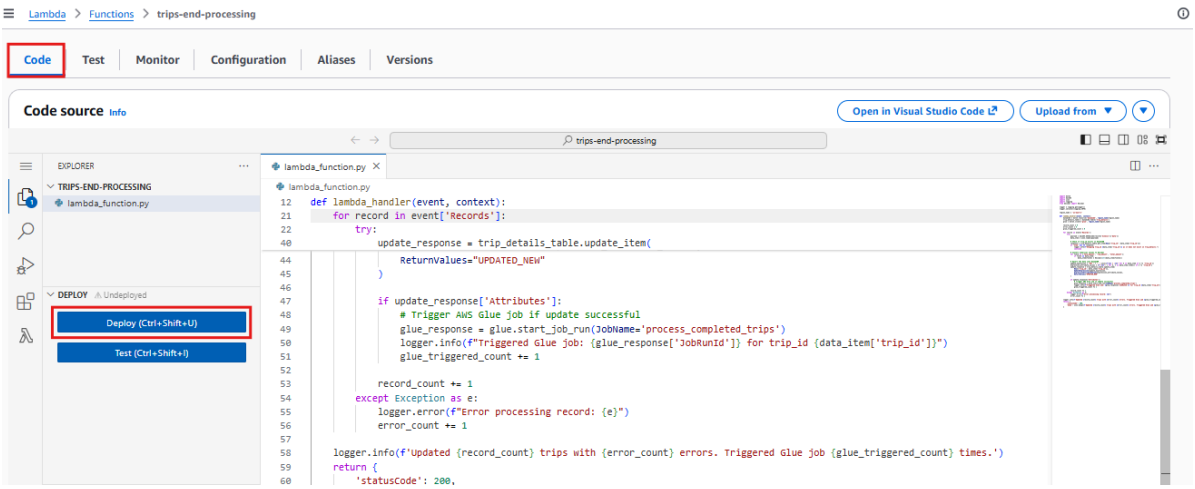
7. Click Add trigger → `trip-end-stream`.



8. Select Trigger: Kinesis —→ Select the stream: trip_end_stream —→ Click Add

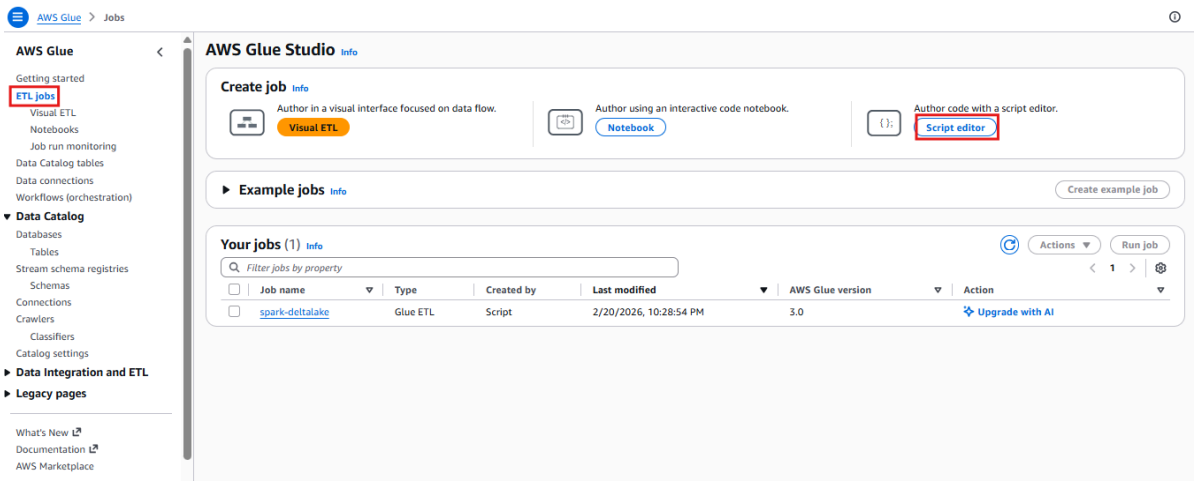


9. Go to code —→ Click Deploy.

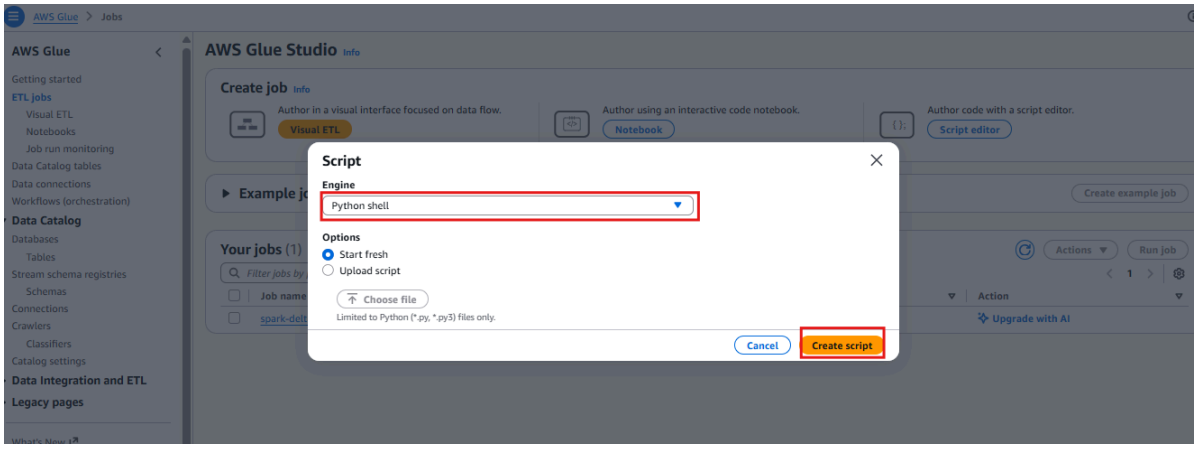


Task 7 — Create Glue Job

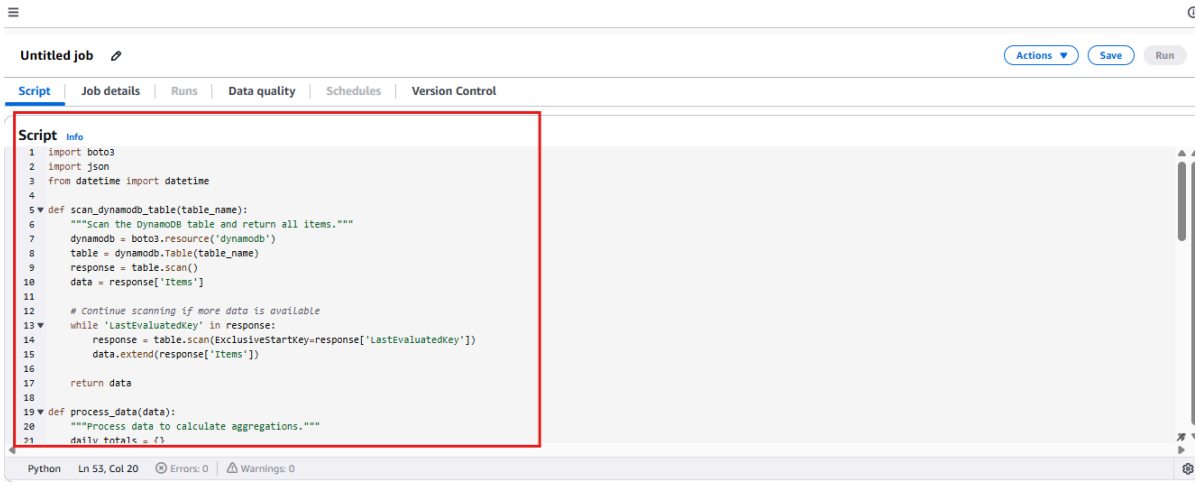
1. Open Glue → ETL Jobs → Script editor.



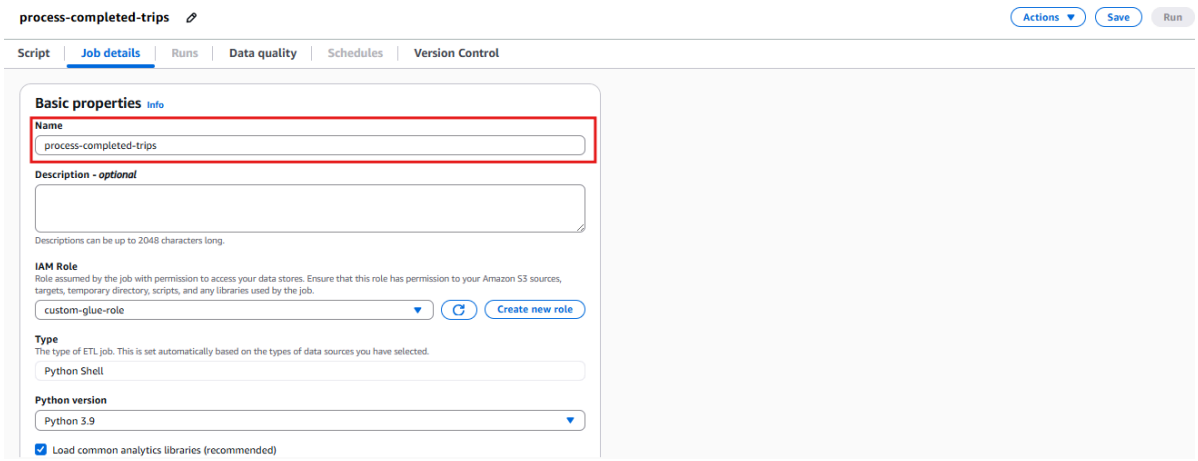
2. Choose **Python Shell**. —→ create script



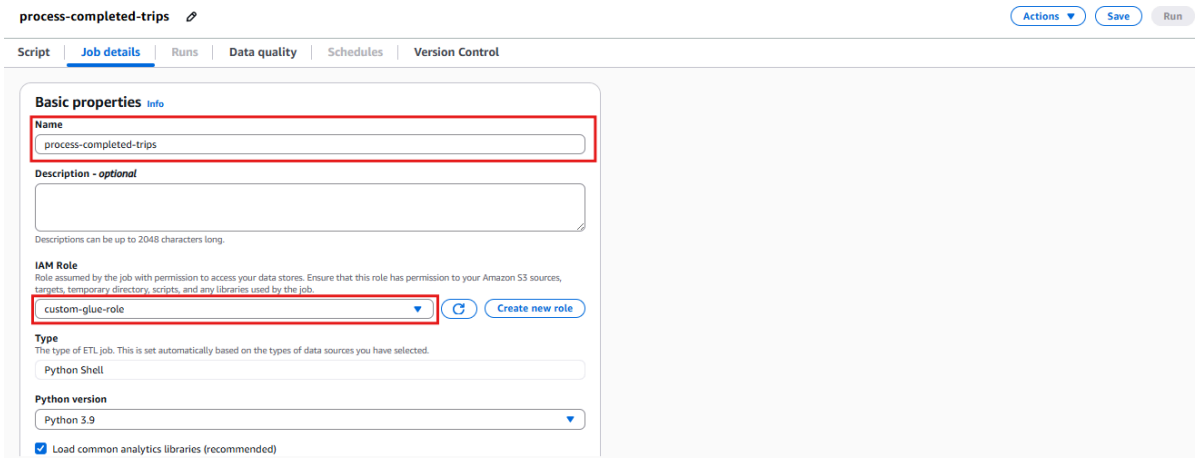
3. Paste glue script.



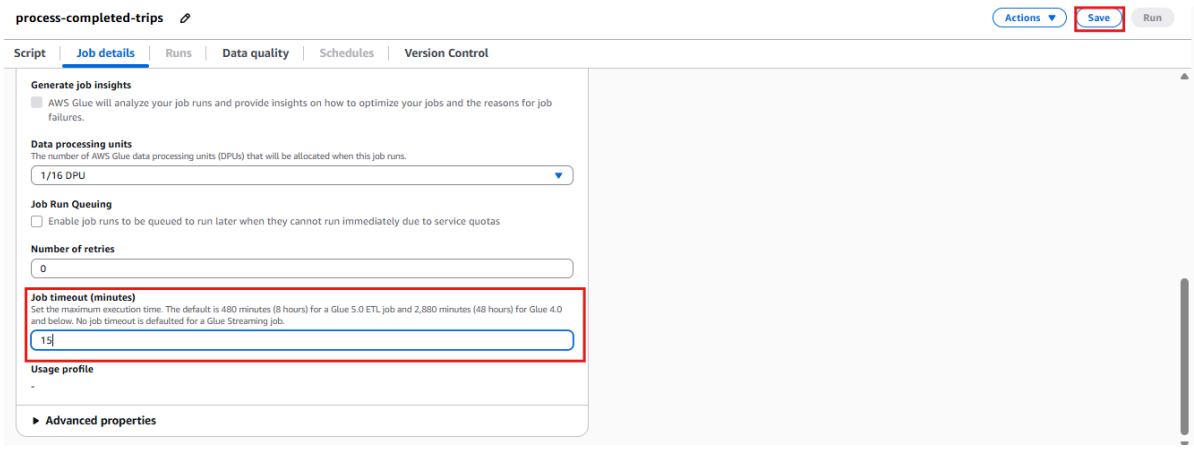
4. Job name: `process-completed-trips`.



5. Select Glue IAM role with DynamoDB + S3 access.

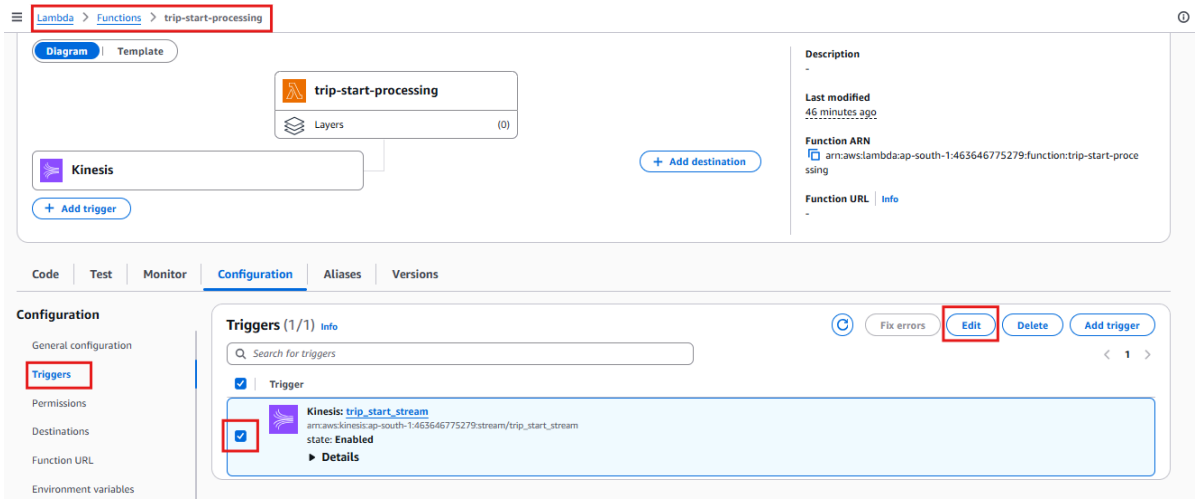


6. Timeout: 15 minutes. —→ Save job.

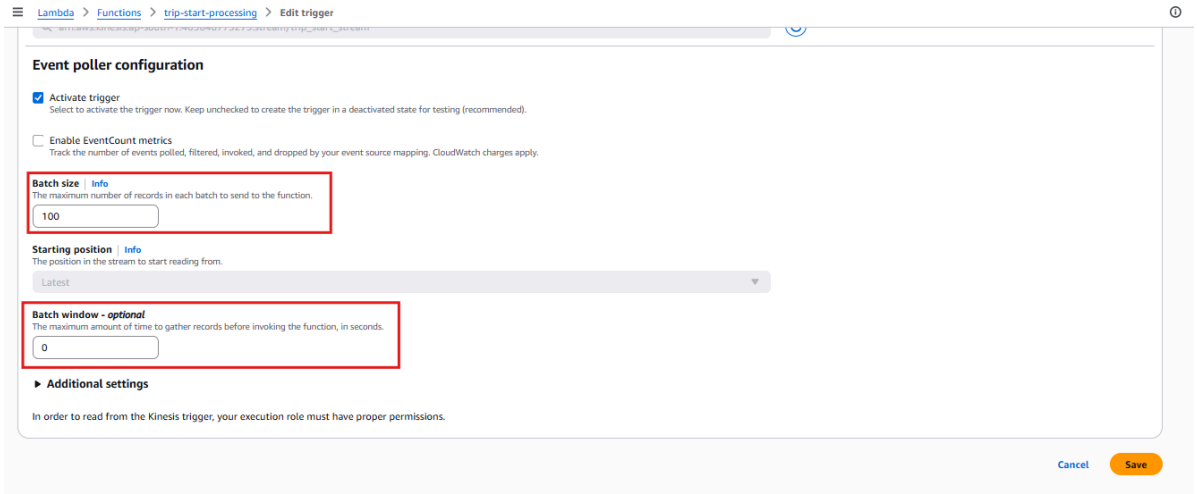


Task 8 — Configure Lambda Batch Settings

1. Open Lambda → Trigger → Edit.



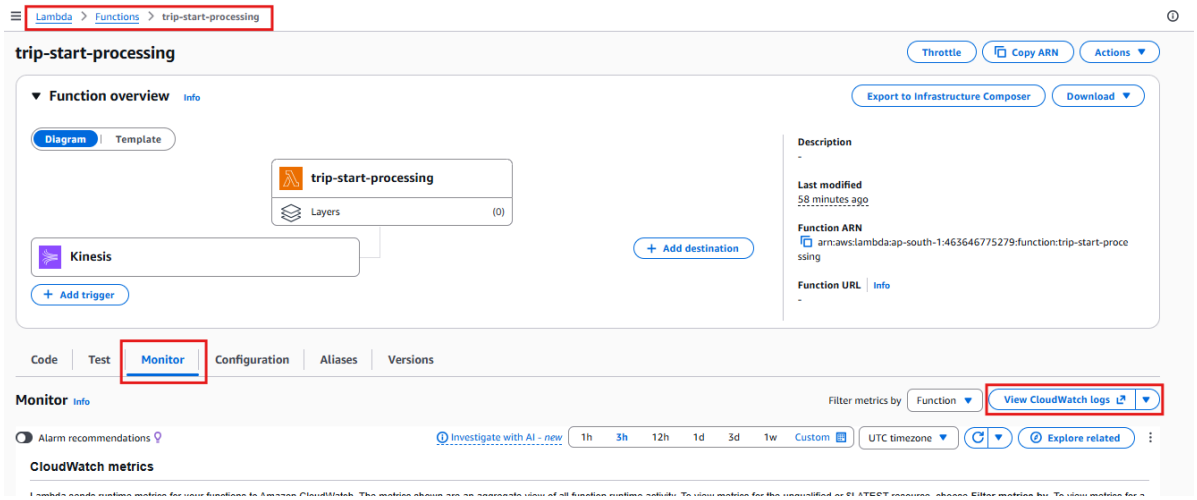
2. Review batch size and batch window.

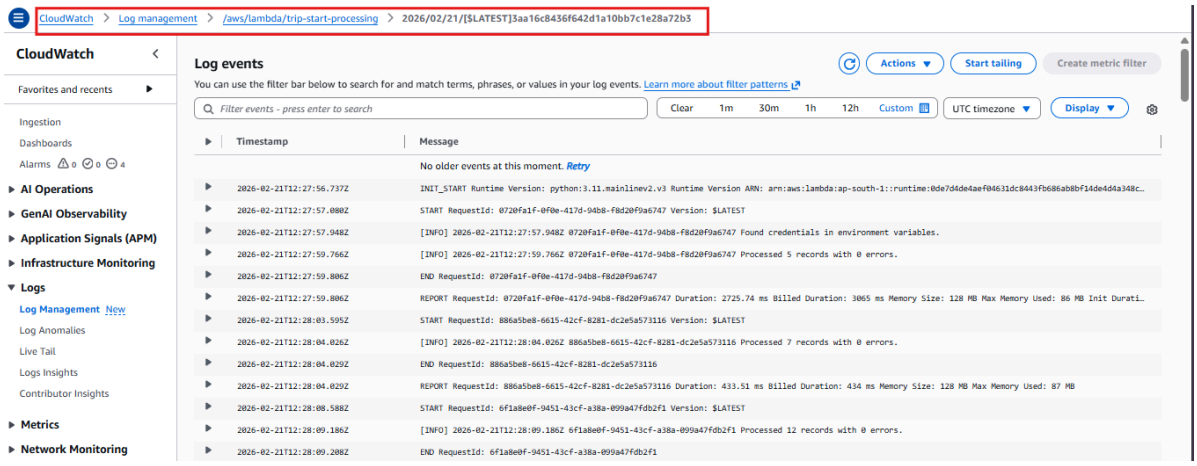


3. Keep defaults or adjust based on data volume.

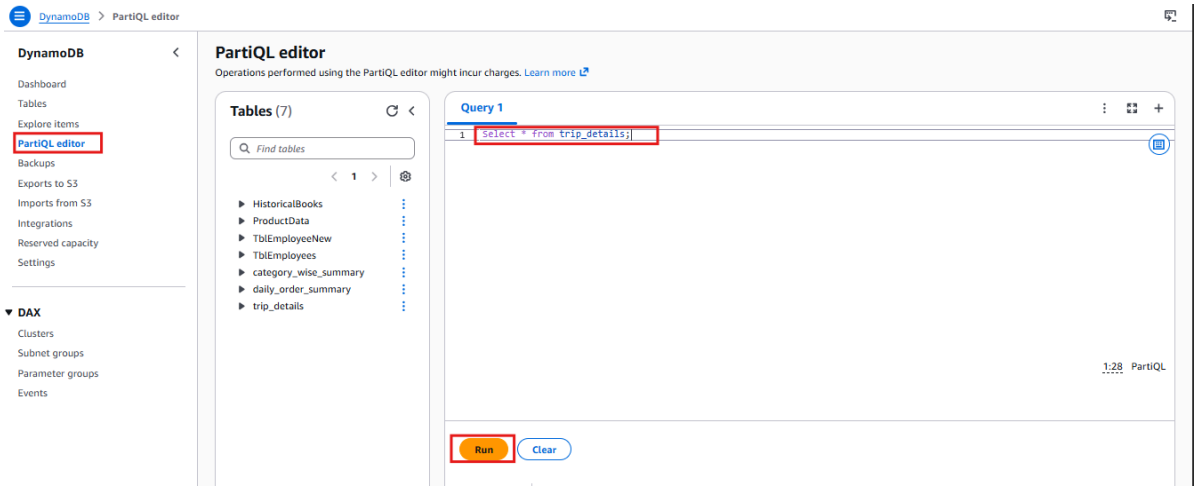
Task 9 — Trigger End-to-End Workflow

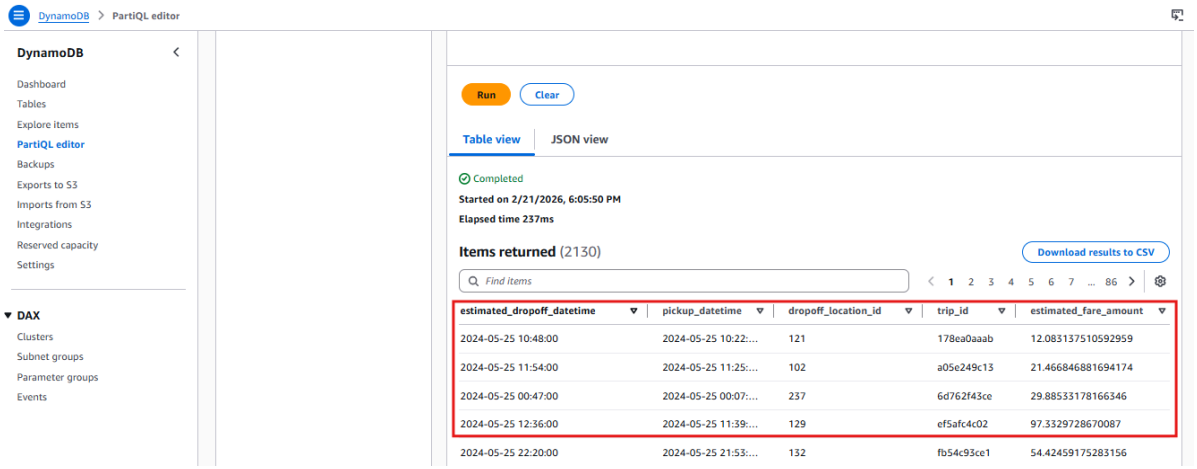
1. Run producer script again.
2. Click on the lambda function —→ Monitor Lambda logs via CloudWatch.





3. Verify DynamoDB table records.





4. Confirm Glue job execution.
5. Check S3 output bucket for aggregated JSON file.

Output

After completing the lab, you should observe:

✅ Trip start events stored in DynamoDB

✅ Trip end events updating existing records

✅ Lambda functions triggered automatically

✅ Glue job aggregating completed trips

✅ Aggregated JSON results stored in S3

✅ CloudWatch logs showing successful processing